

Modul 05: Event-Sourcing

Systemvoraussetzungen: Visual Studio, .NET

Ziel: Implementierung eines einfachen Bankkontos als Konsolenanwendung, wobei die Transaktionen mittels Event-Sourcing im Speicher persistiert werden.

Hinweise: Bei diesen Labs geht es nicht um die vollständige Implementierung, sondern ein Verständnis der Konzepte zu entwickeln. Bitte begründen Sie Ihre Entscheidungen und gewählte Designoptionen. Die Entwürfe können in der Feedbackrunde diskutiert werden.

1 Event-Definitionen

```
public abstract class Event
{
    public Guid AggregateId { get; set; }
    public DateTime Timestamp { get; set; } = DateTime.UtcNow;
}

// TODO: Implementieren Sie folgende Events:
// - AccountOpenedEvent (Mit Initialdepot)
// - MoneyDepositedEvent
// - MoneyWithdrawnEvent
// - AccountClosedEvent
```

2 Aggregat-Implementierung

```
public class BankAccount
{
    private decimal _balance;
    public IList<Event> Changes { get; } = new List<Event>();

    public void Apply(Event @event)
    {
        switch (@event)
        {
            case AccountOpenedEvent e:
                _balance = e.InitialBalance;
                break;
            // TODO: Implementieren Sie die anderen Cases
        }
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException();
        Apply(new MoneyDepositedEvent { Amount = amount });
    }

    // TODO: Implementieren Sie Withdraw und Close-Methoden
}
```

3 Event Store & Replay

```
public class EventStore
{
    private readonly Dictionary<Guid, List<Event>> _store = new();

    public void Save(Guid aggregateId, IEnumerable<Event> events)
    {
        // TODO: Events anfügen
    }

    public BankAccount Load(Guid accountId)
    {
        // TODO: Events materialisieren
    }
}
```

4 Implementierung testen

Eine einfache Konsolenanwendung soll die implementierte Logik testen. Der Benutzer w

```
Console.OutputEncoding = System.Text.Encoding.UTF8;
```

```
while (true)
{
    Console.WriteLine(@"
1. Konto eröffnen
2. Einzahlen
3. Abheben
4. Kontoauszug
5. Konto schließen
0. Beenden");
    var input = Console.ReadLine();

    // TODO: Implementieren Sie die Befehlsverarbeitung
    // Beispiel: Neues Konto mit EventStore.Save(...)
}
```

5 Bonusaufgaben

- Dispokredit als neue Geschäftsregel hinzufügen
- Letzte Transaktion stornierbar machen